



Mastering Datetime in Snowflake SQL

A Guide for Data Engineers

Datetime Data Types in Snowflake

Datetime types are crucial for data engineers as they affect storage, query performance, and time zone handling. Snowflake offers several specialized datetime types:

DATE - Stores dates without time elements (YYYY-MM-DD)

TIME - Stores times in HH:MI:SS format with optional precision

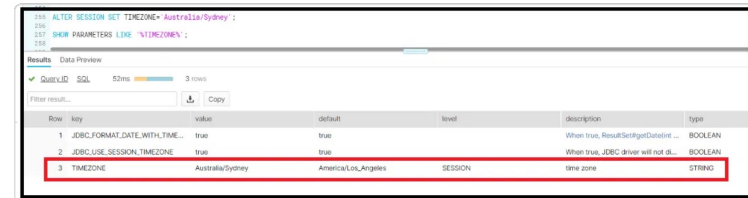
TIMESTAMP_NTZ - "No Time Zone" timestamp, stores "wallclock" time

TIMESTAMP_LTZ - "Local Time Zone" timestamp, stores UTC internally but displays in session time zone

TIMESTAMP_TZ - "With Time Zone" timestamp, stores UTC time with time zone offset

Key Considerations:

Choose **TIMESTAMP_LTZ** for most use cases where time zone awareness is needed. Use **TIMESTAMP_NTZ** when time zones don't matter or for performance-critical operations.



Row	key	value	default	level	description	type
1	JDBC_FORMAT_DATE_WITH_TIME...	true	true		When true, ResultSet.getDateint...	BOOLEAN
2	JDBC_USE_SESSION_TIMEZONE	true	true		When true, JDBC driver will not d...	BOOLEAN
3	TIMEZONE	Australia/Sydney	America/Los_Angeles	SESSION	time zone	STRING

Working with EPOCH and Unix Timestamps

EPOCH/Unix timestamps represent time as seconds elapsed since January 1, 1970, UTC. They're commonly used in APIs, logs, and cross-platform applications.

Snowflake EPOCH Support:

Snowflake supports conversions based on:

Seconds (10-digit)

Milliseconds (13-digit)

Microseconds (16-digit)

Nanoseconds (19-digit)

Data engineers frequently need to convert between EPOCH timestamps and human-readable formats when integrating data from various sources.

EPOCH to Datetime:

```
-- Convert EPOCH seconds to timestamp
SELECT TO_TIMESTAMP (1625097600 );
-- Result: 2021-07-01 00:00:00.000

-- Convert EPOCH milliseconds
SELECT TO_TIMESTAMP (1625097600000 ::number, 3);
-- Result: 2021-07-01 00:00:00.000
```

Datetime to EPOCH:

```
-- Convert to EPOCH seconds
SELECT DATE_PART ('epoch_second' , '2021-07-01' ::timestamp );
-- Result: 1625097600

-- Convert to EPOCH milliseconds
SELECT DATE_PART ('epoch_millisecond' , '2021-07-01'::timestamp );
-- Result: 1625097600000
```

Time Zone Conversions in Snowflake

Time zone handling is critical for global applications and data pipelines. Snowflake provides powerful functions to manage time zone conversions.

Working with UTC:

```
-- Convert local time to UTC
SELECT CONVERT_TIMEZONE (
  'America/Chicago' ,
  'UTC',
  '2023-07-15 14:30:00' ::TIMESTAMP_NTZ
)AS utc_time;

-- Store timestamps in UTC, display in local
CREATE TABLE events (
  event_time TIMESTAMP_LTZ
);
INSERT INTO events VALUES ('2023-07-15 19:30:00 UTC' );
```

Key Time Zone Functions:

CONVERT_TIMEZONE - Converts timestamps between time zones

TIMEZONE - Session parameter that controls default time zone

CURRENT_TIMESTAMP - Returns current timestamp in session time zone

Important Note:

TIMESTAMP_TZ stores only the time zone offset, not the actual time zone name. This matters for daylight saving time calculations.

Time Zone Conversion Examples:

```
-- Set session time zone
ALTER SESSION SET TIMEZONE = 'America/New_York' ;

-- Convert between time zones
SELECT CONVERT_TIMEZONE (
  'America/Los_Angeles' ,
  'America/New_York' ,
  '2023-01-01 20:00:00' ::TIMESTAMP_NTZ
)AS east_coast_time;

-- Result: 2023-01-01 23:00:00.000
```

Date and Time Formatting

Formatting date and time values is essential for data presentation and reporting. Snowflake provides powerful functions for customizing datetime output formats.

Common Format Elements:

Format	Description	Example
YYYY	4-digit year	2025
MM	2-digit month	07
DD	2-digit day	22
HH24	24-hour format	13
MI	Minutes	30
SS	Seconds	45

Session parameters control default formats:

- DATE_OUTPUT_FORMAT
- TIME_OUTPUT_FORMAT
- TIMESTAMP_*_OUTPUT_FORMAT

Formatting Examples:

```
-- Format date as MM/DD/YYYY
SELECT TO_VARCHAR (CURRENT_DATE (), 'MM/DD/YYYY' );
-- Result: 07/22/2025

-- Format with day name and month name
SELECT TO_VARCHAR (CURRENT_DATE (), 'DY, MON DD, YYYY' );
-- Result: TUE, JUL 22, 2025

-- Format timestamp with time
SELECT TO_VARCHAR (CURRENT_TIMESTAMP (),
'YYYY-MM-DD HH24:MI:SS' );
-- Result: 2025-07-22 13:30:45

-- Set session parameter
ALTER SESSION SET DATE_OUTPUT_FORMAT = 'MM/DD/YYYY';

-- Custom format with milliseconds
SELECT TO_VARCHAR (CURRENT_TIMESTAMP (),
'YYYY-MM-DD HH24:MI:SS.FF3 TZh:TzM' );
-- Result: 2025-07-22 13:30:45.123 -07:00
```

Extracting Date and Time Components

Data engineers often need to extract specific components from datetime values for analysis, aggregation, or transformation purposes.

Key Extraction Functions:

DATE_PART(part, date) - Extracts a specific part

EXTRACT(part FROM date) - Alternative syntax

YEAR(), MONTH(), DAY() - Shorthand functions

Common use cases:

Aggregating data by year, quarter, month, or day

Analyzing trends by day of week

Creating custom date hierarchies

Common Extraction Examples:

-- Extract year from a date

```
SELECT DATE_PART ('year', '2023-07-15' ::date);
```

-- Result: 2023

-- Extract month from a date

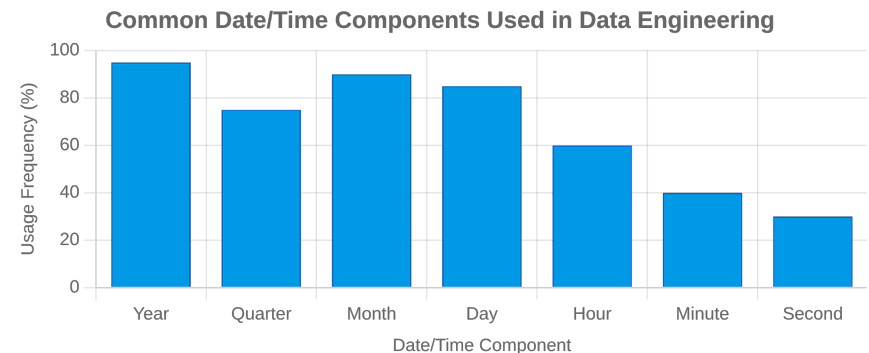
```
SELECT MONTH ('2023-07-15' ::date);
```

-- Result: 7

-- Extract day of week (0=Sun, 6=Sat)

```
SELECT DAYOFWEEK ('2023-07-15' ::date);
```

-- Result: 6 (Saturday)



Date and Time Calculations

Date and time calculations are essential for data analysis, reporting, and ETL processes. Snowflake provides powerful functions for date arithmetic:

Key Date Calculation Functions:

DATEADD - Adds or subtracts a specified time interval

DATEDIFF - Calculates difference between dates in specified units

TIMEDIFF - Alias for DATEDIFF

TIMESTAMPADD - Alias for DATEADD

Common use cases include:

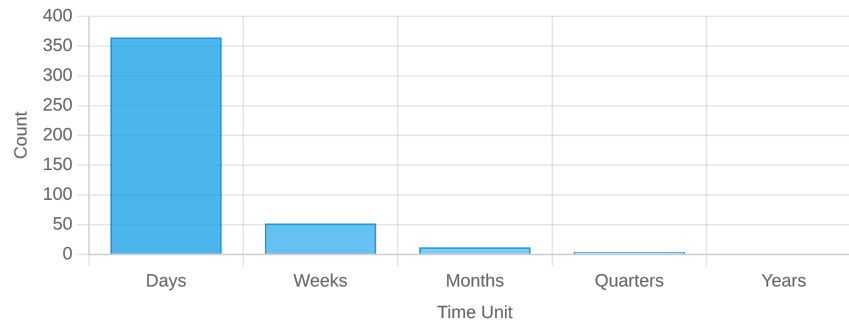
Calculating business days

Finding date ranges (last 30 days, YTD, etc.)

Aging analysis (e.g., accounts receivable)

Time-based SLA calculations

Units in a Year



Adding/Subtracting Time:

-- Add 3 months to a date

```
SELECT DATEADD ('month', 3, '2023-01-15' ::date);
```

-- Result: 2023-04-15

-- Subtract 7 days

```
SELECT DATEADD ('day', -7, CURRENT_DATE ());
```

-- Result: Date 7 days ago

Calculating Date Differences:

-- Days between two dates

```
SELECT DATEDIFF ('day', '2023-01-01' ::date, '2023-02-15' ::date);
```

-- Result: 45

-- Months between dates

```
SELECT DATEDIFF ('month', '2022-03-15' ::date, '2023-03-01' ::date);
```

-- Result: 11

-- Business days calculation

```
SELECT DATEDIFF ('day', '2023-01-01', '2023-01-31') -  
DATEDIFF ('week', '2023-01-01', '2023-01-31') * 2;
```

-- Approximates business days by removing weekends

Best Practices and Common Pitfalls

Best Practices:

- ✓ Use **TIMESTAMP_LTZ** for most use cases to ensure consistent time zone handling
- ✓ Always specify time zone when loading data with timestamps
- ✓ Set **TIMEZONE** session parameter consistently across your organization
- ✓ Use **DATE_TRUNC** to standardize timestamps for aggregations
- ✓ Document your datetime handling approach for team consistency

```
-- Set consistent timezone for your session
ALTER SESSION SET TIMEZONE = 'America/New_York';

-- Standardize timestamps for aggregation
SELECT DATE_TRUNC ('DAY', event_time),
COUNT (*)
FROM events
GROUP BY DATE_TRUNC ('DAY', event_time);
```

Common Pitfalls:

- ⚠ Mixing timestamp types without explicit conversions
- ⚠ Ignoring Daylight Saving Time transitions in calculations
- ⚠ Assuming **TIMESTAMP_TZ** stores the actual time zone (it only stores offset)
- ⚠ Using string literals without explicit type casting
- ⚠ Forgetting to handle NULL values in datetime calculations

```
-- INCORRECT: Mixing timestamp types
SELECT * FROM table1 JOIN table2
ON table1.ts_ltz = table2.ts_ntz;  -- Implicit conversion!

-- CORRECT: Explicit conversion
SELECT * FROM table1 JOIN table2
ON table1.ts_ltz = CONVERT_TIMEZONE ('UTC', table2.ts_ntz);
```




Snowflake Connection Methods

Exploring Python-based Connectivity Options

Introduction



Why Connect to Snowflake?

-  Access and manipulate data stored in Snowflake's cloud data platform
-  Integrate Snowflake with existing applications and workflows
-  Perform data analysis and visualization using familiar tools
-  Build data pipelines and ETL processes
-  Develop machine learning models using Snowflake data

Connection Methods Overview

Snowflake Connector for Python

Direct Python interface for Snowflake operations

SQLAlchemy with Snowflake

ORM-based approach using Python's SQLAlchemy

Snowpark

Advanced API for in-database processing with DataFrame support

Snowflake Connector for Python



What is it?

The Snowflake Connector for Python provides a direct interface for developing Python applications that can connect to Snowflake and perform standard database operations.

Key Features & Use Cases

- ✓ Conforms to Python DB API 2.0 specification
- ✓ Supports authentication methods (key pair, OAuth, etc.)
- ✓ Handles data type conversions between Python and Snowflake
- ✓ Enables parameterized queries for security
- ✓ Supports asynchronous queries and result fetching

```
import snowflake.connector

# Connect to Snowflake
conn = snowflake.connector.connect(user='username',
                                   password='password',
                                   account='account_identifier',
                                   warehouse='warehouse_name',
                                   database='database_name',
                                   schema='schema_name' )

# Create a cursor object
cur = conn.cursor()
# Execute a query
cur.execute("SELECT * FROM my_table")
# Fetch results for row in cur:
print(row)
# Close the connection
conn.close()
```

Pros & Cons

Pros

- + Simple to use
- + Direct SQL control
- + Well-documented

Cons

- Raw SQL required
- No ORM features
- Data processing outside Snowflake

SQLAlchemy with Snowflake



What is SQLAlchemy?

SQLAlchemy is a SQL toolkit and Object-Relational Mapping (ORM) library for Python that provides a full suite of well-known enterprise-level persistence patterns.

Integration with Snowflake

Snowflake SQLAlchemy runs on top of the Snowflake Connector for Python as a dialect to bridge Snowflake database and SQLAlchemy applications.

```
from sqlalchemy import create_engine

engine = create_engine(
    'snowflake://{user}:{password}@{account_identifier}/'.format(
        user='<user_login_name>',
        password='<password>',
        account_identifier='<account_identifier>',
    )
)
try:
    connection = engine.connect()
    results = connection.execute('select current_version()').fetchone()
    print(results[0])
finally:
    connection.close()
    engine.dispose()
```

Key Features & Use Cases

- ✓ Object-Relational Mapping capabilities
- ✓ SQL expression language for complex queries
- ✓ Schema reflection and generation
- ✓ Connection pooling for performance
- ✓ Integration with popular web frameworks

Pros & Cons

Pros

- + ORM capabilities
- + Abstraction from raw SQL
- + Familiar to Python developers

Cons

- Learning curve for ORM
- Performance overhead
- Data processing outside Snowflake

Introducing SnowPark



What is SnowPark?

SnowPark is a set of libraries and runtimes in Snowflake that securely deploy and process non-SQL code, including Python, Java, and Scala.

Key Concept

SnowPark brings computation to the data rather than bringing data to the computation, enabling more efficient data processing directly within Snowflake's environment.

Core Capabilities

- </> Write code in Python, Java, or Scala
- 📊 Process data directly within Snowflake
- 🗃️ Use familiar DataFrame APIs
- ⚡ Leverage Snowflake's scalable compute

Evolution of Snowflake Connectivity

- 1 Early Days**
ODBC/JDBC drivers and native connectors for basic data access
- 2 Python Connector**
Direct Python interface with DB API 2.0 compliance
- 3 SQLAlchemy Integration**
ORM capabilities for more Pythonic data access
- 4 SnowPark Introduction**
In-database processing with DataFrame APIs and multi-language support
- 5 Future Direction**
AI/ML integrations and enhanced developer experience



SnowPark Key Features

</> Supported Languages

SnowPark provides APIs for multiple programming languages, allowing developers to work in their preferred environment.

Python

Java

Scala

🗄️ In-Database Processing

Process data directly within Snowflake's engine, eliminating the need to move data between systems.

This approach significantly improves performance and reduces data transfer costs.

🗃️ DataFrames API

Work with familiar DataFrame abstractions similar to pandas or Spark, making data manipulation intuitive.

Operations are lazily evaluated and optimized by Snowflake's query engine.

```
from snowflake.snowpark import Session

# Define your Snowflake connection parameters
connection_parameters = {
    "account": "<your_snowflake_account>",
    "user": "<your_username>",
    "password": "<your_password>",
    "role": "<your_role>",
    "warehouse": "<your_warehouse>",
    "database": "<your_database>",
    "schema": "<your_schema>"
}

# Create a session
session = Session.builder.configs(connection_parameters).create()

# Create a Snowpark DataFrame from input data
df = session.create_dataframe([[1, 2], [3, 4]], schema=["A", "B"])

# Filter the DataFrame
filtered_df = df.filter(df["A"] > 1)

# Show the results
filtered_df.show()
```

SnowPark vs. Connector/SQLAlchemy

Comparing the three Python-based methods for connecting to and working with Snowflake:

Criteria	 Python Connector	 SQLAlchemy	 SnowPark
Primary Use Case	Direct SQL execution and basic data operations	ORM-based database interactions	Complex data transformations and analytics
Data Processing Location	Client-side (Python)	Client-side (Python)	Server-side (Snowflake)
DataFrame Support	✗ Not native (requires pandas)	✗ Not native (requires pandas)	✓ Native
SQL Knowledge Required	✓ High	🕒 Medium	🕒 Low to Medium
Performance for Large Data	🕒 Limited by client resources	🕒 Limited by client resources	✓ Optimized (server-side)
Learning Curve	✓ Low (if SQL familiar)	🕒 Medium	🕒 Medium
Integration with ML/AI	🕒 Manual	🕒 Manual	✓ Streamlined



Benefits of SnowPark

Simplified Data Pipelines

SnowPark streamlines data pipeline development with a unified API.

- ✔ Write code in familiar languages (Python, Java, Scala)
- ✔ Reduce complexity with DataFrame abstraction
- ✔ Simplify ETL workflows with built-in functions

Enhanced Performance

SnowPark optimizes execution by pushing processing to Snowflake's engine.

- ✔ Leverage Snowflake's distributed processing
- ✔ Automatic query optimization
- ✔ Parallel execution of complex operations

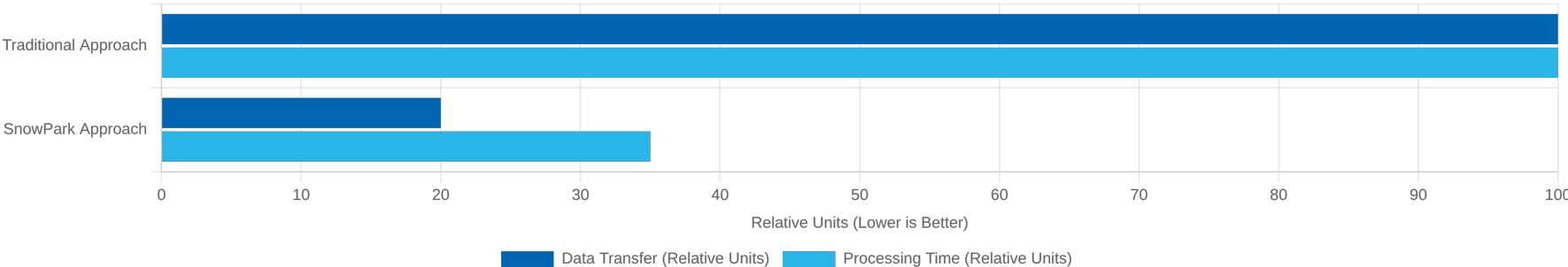
Reduced Data Movement

SnowPark minimizes data transfer between Snowflake and client applications.

- ✔ Process data where it resides
- ✔ Lower network bandwidth requirements
- ✔ Improved security with less data exposure

SnowPark Optimization Flow

Performance Comparison



Conclusion



Summary of Connection Options



Python Connector

Direct interface for SQL operations with DB API 2.0 compliance. Best for simple data access and SQL-focused workflows.



SQLAlchemy

ORM-based approach for more Pythonic data access. Ideal for applications requiring object-relational mapping capabilities.



SnowPark

Advanced in-database processing with DataFrame APIs. Optimal for complex data transformations and analytics at scale.

Future of Snowflake Connectivity

-  Enhanced AI/ML integration with SnowPark, enabling more sophisticated data science workflows directly within Snowflake
-  Expanded support for specialized compute resources, including GPU acceleration for machine learning workloads
-  Improved developer experience with more comprehensive SDKs and tooling across multiple programming languages
-  Greater ecosystem integration with popular data science and analytics frameworks
-  Enhanced security features and governance capabilities for enterprise-grade data applications